

1. What is a JOIN?

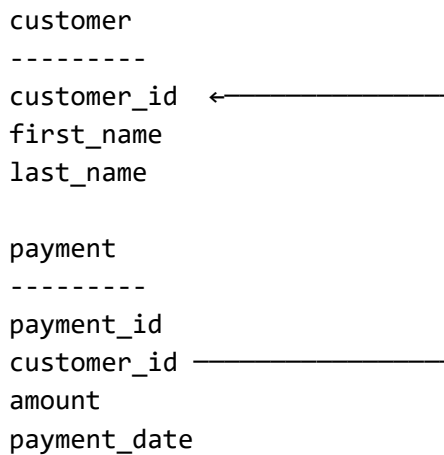
In a database, information is usually divided into different tables.

For example, in the **DVD Rental database**:

- **customer** stores customer information.
- **payment** stores payment information.
- **rental** stores rental information.
- **film** stores film information.
- **inventory** stores available copies of films.
- **staff** stores staff information.

These tables are related through **keys**.

For example:



A **JOIN** allows us to combine related information from two or more tables.

Simple definition

JOIN is used to retrieve related data from multiple tables.

2. Why Do We Use JOIN?

Suppose we run:

```
SELECT *
FROM customer;
```

We get customer information.

If we run:

```
SELECT *  
FROM payment;
```

We get payment information.

But suppose we want to know:

Which customer made each payment?

The `payment` table contains `customer_id`, but we also need the customer's name from the `customer` table.

We can JOIN the tables:

```
SELECT  
    customer.first_name,  
    customer.last_name,  
    payment.amount  
FROM customer  
JOIN payment  
    ON customer.customer_id = payment.customer_id;
```

Result will look conceptually like:

first_name	last_name	amount
Mary	Smith	7.99
Patricia	Johnson	4.99
Linda	Williams	2.99

The JOIN connects:

```
customer.customer_id  
    =  
payment.customer_id
```

3. Understanding Keys Before JOIN

Students should understand these two terms:

Primary Key

A **Primary Key (PK)** uniquely identifies a record.

Example:

```
customer
-----
customer_id  PK
first_name
last_name
```

`customer_id` uniquely identifies each customer.

Foreign Key

A **Foreign Key (FK)** refers to a primary key in another table.

Example:

```
payment
-----
payment_id
customer_id  FK
amount
```

Here:

```
payment.customer_id
      ↓
customer.customer_id
```

This relationship allows us to JOIN the tables.

4. Basic JOIN Syntax

```
SELECT columns
FROM table1
JOIN table2
    ON table1.column = table2.column;
```

Example:

```
SELECT
    customer.first_name,
    customer.last_name,
    payment.amount
FROM customer
JOIN payment
    ON customer.customer_id = payment.customer_id;
```

5. Types of JOINS

The important JOIN types in PostgreSQL are:

1. **INNER JOIN**
2. **LEFT JOIN**
3. **RIGHT JOIN**
4. **FULL OUTER JOIN**
5. **CROSS JOIN**
6. **SELF JOIN**
7. **NATURAL JOIN**

DVD Rental database will be used for practical examples.

6. Table Aliases

An alias gives a table a short name.

Instead of:

```
customer.customer_id
```

we can write:

```
c.customer_id
```

Example:

```
SELECT
    c.first_name,
    c.last_name,
    p.amount
FROM customer AS c
JOIN payment AS p
    ON c.customer_id = p.customer_id;
```

AS is optional:

```
FROM customer c
JOIN payment p
```

Both are valid.

Why use aliases?

Aliases make JOIN queries:

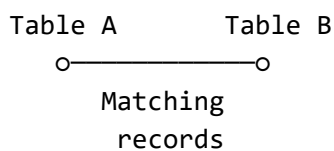
- shorter
- easier to read
- easier to write
- especially useful when multiple tables have columns with the same name

7. INNER JOIN

What is INNER JOIN?

An **INNER JOIN** returns only records that have a matching record in both tables.

Think:



Example

Find customers and their payments:

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    p.amount
FROM customer AS c
INNER JOIN payment AS p
    ON c.customer_id = p.customer_id;
```

What happens?

PostgreSQL finds:

```
customer.customer_id
=
payment.customer_id
```

and combines matching rows.

Important

```
JOIN
```

and

```
INNER JOIN
```

normally mean the same thing.

So this:

```
FROM customer
JOIN payment
  ON customer.customer_id = payment.customer_id
```

is equivalent to:

```
FROM customer
INNER JOIN payment
  ON customer.customer_id = payment.customer_id
```

8. Practical INNER JOIN Example

Question

Show the first name, last name, and payment amount of customers.

```
SELECT
  c.first_name,
  c.last_name,
  p.amount
FROM customer c
INNER JOIN payment p
  ON c.customer_id = p.customer_id;
```

Add a WHERE condition

Find payments greater than 5:

```
SELECT
  c.first_name,
  c.last_name,
  p.amount
FROM customer c
```

```
INNER JOIN payment p
  ON c.customer_id = p.customer_id
WHERE p.amount > 5;
```

This demonstrates that **JOIN** and **WHERE** can be used together.

9. Joining Three Tables

JOIN is not limited to two tables.

Suppose we want:

Customer name + rental date + film title

We need:

```
customer
  ↓
rental
  ↓
inventory
  ↓
film
```

Query:

```
SELECT
  c.first_name,
  c.last_name,
  r.rental_date,
  f.title
FROM customer c
JOIN rental r
  ON c.customer_id = r.customer_id
JOIN inventory i
  ON r.inventory_id = i.inventory_id
JOIN film f
  ON i.film_id = f.film_id;
```

This is a very useful real-world example.

10. LEFT JOIN

What is LEFT JOIN?

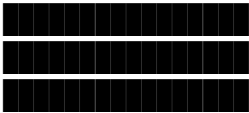
A **LEFT JOIN** returns:

All records from the left table + matching records from the right table.

If there is no match, PostgreSQL returns **NULL** for the right table's columns.

Think:

LEFT TABLE RIGHT TABLE



+
matching

Example

```
SELECT
    c.customer_id,
    c.first_name,
    p.payment_id,
    p.amount
FROM customer c
LEFT JOIN payment p
    ON c.customer_id = p.customer_id;
```

Meaning

Show **all customers**, including customers who do not have a matching payment.

If a customer has no payment:

customer_id	first_name	payment_id	amount
101	John	NULL	NULL

The customer is still shown because **customer** is the **left table**.

11. Why LEFT JOIN Is Useful

Suppose the question is:

Show all customers, including customers who have never made a payment.

Use:

```
SELECT
    c.customer_id,
```

```
c.first_name,  
c.last_name,  
p.payment_id  
FROM customer c  
LEFT JOIN payment p  
ON c.customer_id = p.customer_id;
```

The LEFT JOIN keeps every customer.

12. Find Records with No Match

A very useful technique:

```
SELECT  
    c.customer_id,  
    c.first_name,  
    c.last_name  
FROM customer c  
LEFT JOIN payment p  
    ON c.customer_id = p.customer_id  
WHERE p.payment_id IS NULL;
```

Meaning:

Find customers who have **no payment record**.

This pattern is important:

```
LEFT JOIN ...  
WHERE right_table.id IS NULL
```

It is commonly used to find records without a matching record.

13. RIGHT JOIN

A **RIGHT JOIN** returns:

All records from the right table + matching records from the left table.

Example:

```
SELECT  
    c.first_name,  
    c.last_name,  
    p.payment_id,
```

```
p.amount
FROM customer c
RIGHT JOIN payment p
ON c.customer_id = p.customer_id;
```

The important table here is the **right table**, *payment*.

LEFT JOIN vs RIGHT JOIN

LEFT JOIN

```
FROM customer c
LEFT JOIN payment p
```

Keeps **all customers**.

RIGHT JOIN

```
FROM customer c
RIGHT JOIN payment p
```

Keeps **all payments**.

Simple rule

LEFT JOIN → keep everything from the left table.

RIGHT JOIN → keep everything from the right table.

In practice, many developers prefer LEFT JOIN because it is often easier to read. A RIGHT JOIN can usually be rewritten as a LEFT JOIN by reversing the table order.

14. FULL OUTER JOIN

A **FULL OUTER JOIN** returns:

- matching rows
- unmatched rows from the left table
- unmatched rows from the right table

Conceptually:

LEFT TABLE	RIGHT TABLE
████████████████████	████████████████████
████████████████████	████████████████████


ALL RECORDS

Syntax:

```
SELECT ...  
FROM table1  
FULL OUTER JOIN table2  
    ON table1.column = table2.column;
```

DVD Rental Example

For demonstration:

```
SELECT  
    c.customer_id,  
    c.first_name,  
    p.payment_id,  
    p.amount  
FROM customer c  
FULL OUTER JOIN payment p  
    ON c.customer_id = p.customer_id;
```

This returns all rows from both tables.

Important DVD Rental observation

In the standard DVD Rental database, customers and payments are related through `customer_id`, so most/all payment records normally have a matching customer. Therefore, this example is useful for learning the syntax, but it may not visibly demonstrate unmatched rows.

15. SELF JOIN

15.1. What is a Self Join?

A **Self Join** is a join in which a table is joined with **itself**.

We use a Self Join when we want to compare or connect rows within the **same table**.

Simple Definition

Self Join = Joining a table with itself.

15.2. Employee–Manager Example

Consider an **employee** table:

```
employee
-----
employee_id
employee_name
manager_id
```

Example data:

employee_id	employee_name	manager_id
1	Ali	NULL
2	Ahmed	1
3	Sara	1
4	Bilal	2
5	Ayesha	2

Here:

- **employee_id** identifies an employee.
- **employee_name** stores the employee's name.
- **manager_id** stores the **employee_id** of that employee's manager.
- **NULL** means the employee does not have a manager.

For example:

```
Ahmed → manager_id = 1
```

Employee ID **1** belongs to **Ali**, so Ali is Ahmed's manager.

15.3. Why Do We Need a Self Join?

Both the **employee** and the **manager** are stored in the same table.

We need to connect:

```
Employee's manager_id
      ↓
Manager's employee_id
```

Therefore, we join the **employee** table with itself.

15.4. Basic Self Join Query

```
SELECT
  e.employee_name AS employee,
  m.employee_name AS manager
FROM employee AS e
JOIN employee AS m
  ON e.manager_id = m.employee_id;
```

Example Output

employee	manager
Ahmed	Ali
Sara	Ali
Bilal	Ahmed
Ayesha	Ahmed

15.5. Understand the Query

Step 1: First copy of the table

```
employee AS e
```

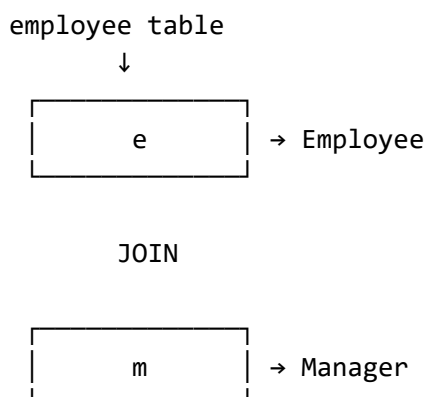
e represents the employee.

Step 2: Second copy of the table

```
employee AS m
```

m represents the manager.

Although both **e** and **m** come from the same table, we give them different aliases so that we can distinguish them.



JOIN



Step 3: Connect employee with manager

```
ON e.manager_id = m.employee_id
```

This means:

Find the employee whose **manager_id** matches the manager's **employee_id**.

15.6. How the Self Join Works

Consider Ahmed:

employee_id	employee_name	manager_id
2	Ahmed	1

Ahmed's **manager_id** is 1.

Now look for:

```
employee_id = 1
```

The table contains:

employee_id	employee_name
1	Ali

Therefore:

```
Ahmed → Ali  
Employee → Manager
```

15.7. Important Point About Table Aliases

We use aliases because the same table appears twice.

```
employee AS e  
employee AS m
```

Here:

- **e** = employee
- **m** = manager

Therefore:

```
e.employee_name
```

means the employee's name.

And:

```
m.employee_name
```

means the manager's name.

15.8. Self Join with Employee ID

We can also display the IDs:

```
SELECT
  e.employee_id,
  e.employee_name AS employee,
  m.employee_id AS manager_id,
  m.employee_name AS manager
FROM employee AS e
JOIN employee AS m
  ON e.manager_id = m.employee_id;
```

Output

employee_id	employee	manager_id	manager
2	Ahmed	1	Ali
3	Sara	1	Ali
4	Bilal	2	Ahmed
5	Ayesha	2	Ahmed

1.9. INNER JOIN vs LEFT JOIN

The previous query uses:

```
JOIN
```

which means **INNER JOIN**.

It does **not** display employees who do not have a manager.

If we want to display **all employees**, including the top-level manager, use **LEFT JOIN**.

```
SELECT
  e.employee_name AS employee,
  m.employee_name AS manager
FROM employee AS e
LEFT JOIN employee AS m
  ON e.manager_id = m.employee_id;
```

Output

employee	manager
Ali	NULL
Ahmed	Ali
Sara	Ali
Bilal	Ahmed
Ayesha	Ahmed

Ali appears because **LEFT JOIN** keeps all employees.

15.10. Key Concept

The most important part of this Self Join is:

```
ON e.manager_id = m.employee_id
```

Think of it as:

```
Employee's manager_id
      ↓
      =
      ↓
Manager's employee_id
```

Remember

Self Join is useful when rows in the same table have a relationship with other rows in that same table.

Common examples include:

- Employee → Manager
- Employee → Supervisor
- Employee → Department Head
- Category → Parent Category
- Employee → Mentor

15.11. Create Employee Table

Use the following PostgreSQL query to create the **employee** table for the Self Join example:

```
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(50),  
    manager_id INT  
);
```

Insert Sample Data

```
INSERT INTO employee (employee_id, employee_name, manager_id)  
VALUES  
    (1, 'Ali', NULL),  
    (2, 'Ahmed', 1),  
    (3, 'Sara', 1),  
    (4, 'Bilal', 2),  
    (5, 'Ayesha', 2);
```

Check the Table

```
SELECT * FROM employee;
```

Result

employee_id	employee_name	manager_id
1	Ali	NULL
2	Ahmed	1
3	Sara	1
4	Bilal	2
5	Ayesha	2

Now you can practice the Self Join:

```
SELECT  
    e.employee_name AS employee,  
    m.employee_name AS manager  
FROM employee AS e  
LEFT JOIN employee AS m  
    ON e.manager_id = m.employee_id;
```

17. CROSS JOIN

A **CROSS JOIN** produces every possible combination of rows from two tables.

Suppose:

Table A = 3 rows
Table B = 4 rows

CROSS JOIN produces:

$3 \times 4 = 12$ rows

Syntax

```
SELECT *  
FROM table1  
CROSS JOIN table2;
```

DVD Rental Example

```
SELECT  
    c.first_name,  
    c.last_name,  
    s.first_name AS staff_first_name  
FROM customer c  
CROSS JOIN staff s;
```

This produces every customer–staff combination.

If there are 599 customers and 2 staff members:

$599 \times 2 = 1198$ rows

Important

A CROSS JOIN does **not** require an **ON** condition.

18. When Would We Use CROSS JOIN?

CROSS JOIN is useful when we intentionally need **all possible combinations**.

For example:

Products × Sizes

could produce:

Shirt + Small
Shirt + Medium
Shirt + Large
Shoes + Small
Shoes + Medium
Shoes + Large

But be careful:

CROSS JOIN can produce a very large number of rows.

19. NATURAL JOIN

A **NATURAL JOIN** automatically joins tables using columns that have the **same name**.

Syntax:

```
SELECT *  
FROM table1  
NATURAL JOIN table2;
```

PostgreSQL automatically looks for columns with matching names.

DVD Rental Example

customer and **payment** both have:

customer_id

So we could write:

```
SELECT  
    first_name,  
    last_name,  
    amount  
FROM customer  
NATURAL JOIN payment;
```

PostgreSQL automatically uses the common column:

```
customer_id
```

20. Why Be Careful with NATURAL JOIN?

NATURAL JOIN can be convenient, but it is generally less explicit.

Suppose today two tables have:

```
customer_id
```

as the only common column.

Later, a new column with the same name is added to both tables.

The behavior of the NATURAL JOIN could change automatically.

Therefore, beginners should generally prefer:

```
JOIN ...  
ON ...
```

because it clearly tells us **which columns are being used for the relationship**.

For example:

```
SELECT  
    c.first_name,  
    c.last_name,  
    p.amount  
FROM customer c  
JOIN payment p  
    ON c.customer_id = p.customer_id;
```

This is much easier to understand.

21. JOIN with WHERE

JOIN and WHERE have different jobs.

JOIN

Connects related tables.

```
ON c.customer_id = p.customer_id
```

WHERE

Filters the result.

```
WHERE p.amount > 5
```

Example:

```
SELECT
    c.first_name,
    c.last_name,
    p.amount
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
WHERE p.amount > 5;
```

Think:

```
JOIN → Connect tables
WHERE → Filter rows
```

22. JOIN with ORDER BY

We can also sort the result.

```
SELECT
    c.first_name,
    c.last_name,
    p.amount
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
ORDER BY p.amount DESC;
```

This shows the highest payment amounts first.

23. JOIN with GROUP BY

We can calculate the total payment made by each customer.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(p.amount) AS total_payment
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
ORDER BY total_payment DESC;
```

This is a very practical example of JOIN + aggregate functions.

24. JOIN + GROUP BY + HAVING

Find customers whose total payments are greater than 100:

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(p.amount) AS total_payment
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
HAVING SUM(p.amount) > 100;
```

Remember:

```
JOIN      → connect tables
WHERE     → filter individual rows
GROUP BY → create groups
HAVING    → filter groups
```

25. JOIN with Multiple Tables — Practical Example

Question

Show the customer name, film title, and rental date.

We need four tables:

```
customer
  ↓
rental
  ↓
inventory
  ↓
film
```

Query:

```
SELECT
    c.first_name,
    c.last_name,
    f.title,
    r.rental_date
FROM customer c
JOIN rental r
    ON c.customer_id = r.customer_id
JOIN inventory i
    ON r.inventory_id = i.inventory_id
JOIN film f
    ON i.film_id = f.film_id;
```

This is one of the best examples for understanding why JOINS are important.

26. Practical Example — Customer + Film + Payment

Suppose we want:

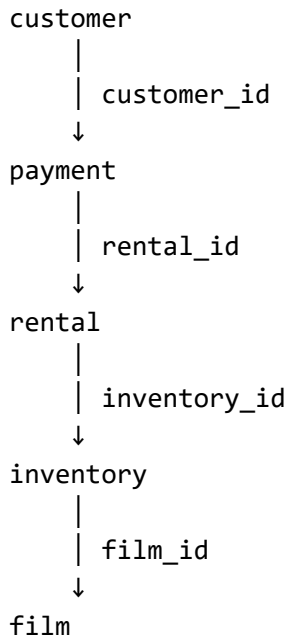
Customer name, film title, payment amount.

We can use:

```
SELECT
    c.first_name,
    c.last_name,
    f.title,
    p.amount
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
JOIN rental r
```

```
    ON p.rental_id = r.rental_id
JOIN inventory i
    ON r.inventory_id = i.inventory_id
JOIN film f
    ON i.film_id = f.film_id;
```

Conceptually:



27. JOIN Types — Quick Comparison

JOIN	What does it return?
INNER JOIN	Matching records from both tables
LEFT JOIN	All left records + matching right records
RIGHT JOIN	All right records + matching left records
FULL OUTER JOIN	All records from both tables
CROSS JOIN	Every possible combination
SELF JOIN	A table joined with itself
NATURAL JOIN	Automatically joins columns with the same names

28. Visual Understanding

INNER JOIN

Table A Table B

AAAAA

BBBBB

↑

MATCH

Only matching records.

LEFT JOIN

Table A Table B

AAAAAAAAA

BBB

All A + matching B.

RIGHT JOIN

Table A Table B

AAA

BBBBBBBBB

Matching A + all B.

FULL OUTER JOIN

Table A Table B

AAAAAAAAA BBBBBBBBB

ALL

Everything from both tables.

CROSS JOIN

A × B

A1 → B1

A1 → B2

A2 → B1

A2 → B2

...

Every combination.

29. The Most Important JOIN Pattern

Students should first master this pattern:

```
SELECT
    table1.column,
    table2.column
FROM table1
JOIN table2
    ON table1.common_column = table2.common_column;
```

For DVD Rental:

```
SELECT
    c.first_name,
    c.last_name,
    p.amount
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id;
```

Then add filtering:

```
WHERE p.amount > 5;
```

Then sorting:

```
ORDER BY p.amount DESC;
```

Complete query:

```
SELECT
    c.first_name,
    c.last_name,
    p.amount
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
WHERE p.amount > 5
ORDER BY p.amount DESC;
```

30. Beginner Rules to Remember

Rule 1 — JOIN connects tables

```
JOIN payment p  
ON c.customer_id = p.customer_id
```

Rule 2 — ON specifies the relationship

```
ON table1.id = table2.id
```

Rule 3 — WHERE filters records

```
WHERE p.amount > 5
```

Rule 4 — LEFT JOIN keeps everything from the left table

```
FROM customer c  
LEFT JOIN payment p
```

Rule 5 — RIGHT JOIN keeps everything from the right table

```
FROM customer c  
RIGHT JOIN payment p
```

Rule 6 — INNER JOIN keeps only matches

```
INNER JOIN
```

Rule 7 — CROSS JOIN creates combinations

```
CROSS JOIN
```

Rule 8 — SELF JOIN joins a table with itself

```
staff s1  
JOIN staff s2
```

Rule 9 — NATURAL JOIN automatically finds same-named columns

NATURAL JOIN

Rule 10 — Prefer explicit JOIN conditions

For beginner and professional SQL, this is usually clearer:

```
JOIN payment p
ON c.customer_id = p.customer_id
```

rather than relying on:

NATURAL JOIN

31. Practice Exercises — DVD Rental

Try these without looking at the answers first.

Exercise 1

Display customer first name, last name, and payment amount.

Exercise 2

Display customer names and payments greater than 5.

Exercise 3

Display all customers and their payments, including customers with no payment.

Exercise 4

Find customers who have no payment records.

Exercise 5

Display customer name, rental date, and film title.

Exercise 6

Display film titles and their rental rates.

Exercise 7

Display each customer's total payment.

Exercise 8

Display customers whose total payment is greater than 100.

Exercise 9

Display all possible customer and staff combinations using **CROSS JOIN**.

Exercise 10

Use a **SELF JOIN** on the **staff** table to find pairs of staff members working at the same store.

Exercise 11

Use **NATURAL JOIN** to join **customer** and **payment**.

Exercise 12

Rewrite the **NATURAL JOIN** query using an explicit **JOIN ... ON** condition.

Final Cheat Sheet

```
-- INNER JOIN
SELECT *
FROM customer c
JOIN payment p
ON c.customer_id = p.customer_id;
```

```
-- LEFT JOIN
SELECT *
FROM customer c
LEFT JOIN payment p
ON c.customer_id = p.customer_id;
```

```
-- RIGHT JOIN
SELECT *
FROM customer c
RIGHT JOIN payment p
ON c.customer_id = p.customer_id;
```

```
-- FULL OUTER JOIN
SELECT *
FROM customer c
FULL OUTER JOIN payment p
ON c.customer_id = p.customer_id;
```

```
-- CROSS JOIN
SELECT *
FROM customer c
CROSS JOIN staff s;
```

```
-- SELF JOIN
SELECT *
FROM staff s1
JOIN staff s2
ON s1.store_id = s2.store_id;
```

```
-- NATURAL JOIN
SELECT *
FROM customer
NATURAL JOIN payment;
```

One-line memory trick

INNER = matching | LEFT = all left | RIGHT = all right | FULL = everything | CROSS = combinations | SELF = same table | NATURAL = same-named columns